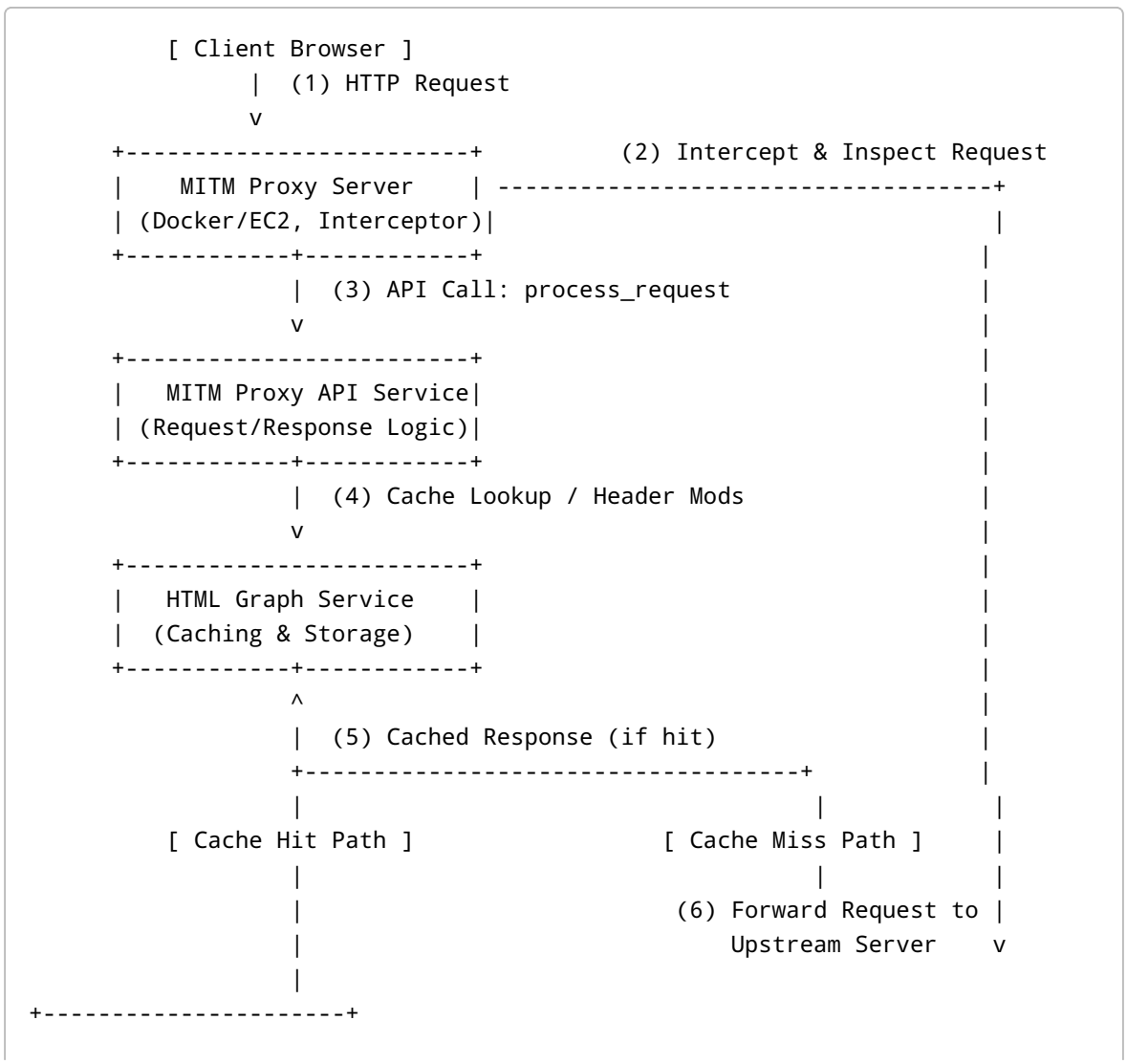
ChatGPT

Introduction

We have developed a **Man-in-the-Middle (MITM) Proxy platform** to intercept and process web traffic in real-time. This system is designed to capture HTTP(S) requests and responses from a client browser, apply custom processing (such as caching and content transformation), and then forward the responses to the user. The primary goal of this platform is to enable advanced content analysis and caching while being transparent to the end-user. This document provides a comprehensive briefing of the platform's architecture and workflow, intended for a technical audience (including our business partners who were present in the meeting). The style is technical and professional, akin to a white paper, detailing how the system works and how its components interact.

System Architecture

Our MITM proxy platform is composed of several key components working in tandem. Below is an overview of the architecture:



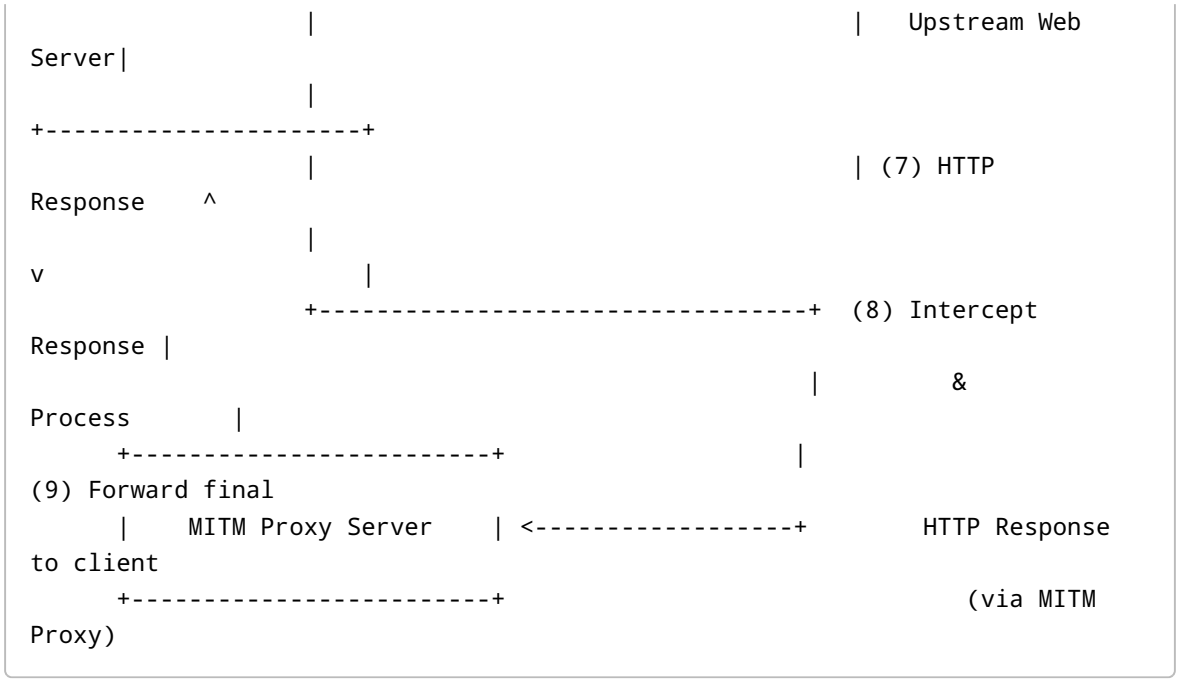


Figure: Architecture and Data Flow of the MITM Proxy Platform. The numbered steps correspond to the workflow described in the next section.

Components

- **Client Browser:** A user's web browser configured to use the MITM proxy as its HTTP/HTTPS proxy. All web requests from the browser are routed through the proxy. The browser trusts the proxy's certificate (for HTTPS) so the proxy can decrypt and re-encrypt traffic as needed (man-in-the-middle of the TLS connection).

- **MITM Proxy Server:** The intercepting proxy component (running on a Docker container or an EC2 instance). It accepts incoming HTTP(S) requests from the browser and mediates all traffic. The proxy is responsible for pausing the traffic at two hook points – **request** and **response** – and delegating processing to the API service. Our proxy implementation is a *battle-hardened, high-performance* service capable of handling production traffic at scale. We can deploy multiple instances of this proxy server on EC2 for scalability.

- **MITM Proxy API Service:** A custom web service (written in Python) that contains the logic for processing intercepted requests and responses. The proxy server communicates with this API service through defined endpoints:

- `process_request` – called when a new request is intercepted (before reaching the upstream server).
- `process_response` – called when the response is received from the upstream server (before returning to the browser).
  This service encapsulates our core custom logic: deciding whether to serve from cache, what to log, how to modify requests/responses, and storing or transforming content. The API service is decoupled from the proxy mechanism; it doesn't need to know whether the call is coming from a real proxy or a simulation, making it versatile for testing.

- **HTML Graph Service (Caching & Storage):** This is a subsystem (or module within the API service) responsible for storing and retrieving web content and related metadata. When the API service decides to cache a page or needs to retrieve a cached page, it interacts with this service. The HTML Graph Service provides methods such as:

- **Save HTML (cache store):** Persist the HTML content of pages (and possibly associated data like headers or a DOM graph representation) into a storage backend.
- **Load HTML (cache fetch):** Retrieve previously cached content for a given URL or identifier.

- It also logs **processing flows** – sequences of actions performed on the content (e.g., caching events, transformations) along with timing and status information. This creates an audit trail or graph of what happened to each piece of content. The storage can be thought of as a database or object store where each web page's HTML (and possibly a parsed structure) is saved, keyed by URL or content hash. The term "Graph" indicates that in the future, the HTML content may be represented or linked as a graph of DOM nodes or linked resources, but at present the service mainly handles flat HTML content and simple metadata.

- **Upstream Web Server:** The external server on the internet that the browser is trying to reach (for example, any public website). The MITM proxy sits between the browser and this upstream server. From the browser's perspective, the proxy *is* the server (for HTTPS, the proxy generates a certificate on-the-fly for the target host), and from the upstream server's perspective, the proxy is the client. The upstream server is unaware of the interception; it responds normally to the proxy's forwarded requests.

- **Administrative UI & Simulation Tools:** In addition to the core services above, we have built a user interface to visualize and test the system's behavior. This includes:

- A **Simulation UI** that allows developers to emulate browser requests and upstream responses without using an actual browser. This tool directly calls the `process_request` and `process_response` methods on the API service with specified inputs. It is invaluable for debugging and developing the logic, as it can simulate high volumes of traffic or edge cases (like various headers, response types, etc.) without manual browser interaction.
- A **Cache Browser/Editor UI** that lets us inspect the content stored in the HTML Graph Service. We can list cached pages, view their HTML content, and even edit the stored content for testing. This UI also displays the recorded flows and performance metrics for each request (e.g., how long each processing step took, what transformations were applied, etc.). It serves as a window into the internal state of the system for monitoring and verification.

Request Handling Workflow

When the browser initiates a web request, the following sequence of events occurs through the MITM proxy system:

1. **Browser Sends Request:** The client browser issues an HTTP/HTTPS request (for example, an HTTP GET for a web page). Because the browser is configured to use the MITM proxy, this request is sent to the **MITM Proxy Server** instead of directly to the target host. The request includes the method (GET, POST, etc.), the destination host and path, headers, and possibly cookies or other relevant metadata. At this stage, the actual request body (for GET there is none, for POST it could be form data) is not yet needed by our logic – we mainly care about the URL and headers for caching purposes.

2. **MITM Proxy Intercepts the Request:** The proxy server receives the request and logs its details (method, URL path, headers, cookies, etc.). It then pauses the forwarding of this request and invokes the `process_request` method on the MITM Proxy API Service. Essentially, the proxy hands over a summary of the request (without contacting the upstream server yet) to our API logic for analysis.

3. **API Service Checks Cache / Pre-processes Request:** The `process_request` handler in the API service examines the incoming request information. Its primary job at this point is to determine if we have a cached response for the requested URL (and relevant context like cookies, if those affect content).

4. **Cache Hit:** If the HTML Graph Service indicates that the exact content has been seen before and is stored in the cache, the API can retrieve that content immediately. In such a case, `process_request` will return a response payload (including the cached HTML body and possibly cached response headers) back to the proxy.
5. **Cache Miss:** If the content is not in cache, `process_request` will not return any body content. Instead, it may provide *instructions* to the proxy on how to proceed. For example, it might add or modify certain outbound request headers (for the upstream request) for debugging or tracking purposes. In our implementation, we have the ability to inject headers (e.g., to signal to the upstream or to carry a trace ID), but this is optional. In most cache miss cases, the API's response is essentially "no cached data – proceed with upstream request."

6. Regardless of hit or miss, this stage has minimal latency. It's a quick lookup and decision. No external calls (besides checking our own cache store) are made yet if it's a miss.

7. **Immediate Response on Cache Hit:** In the cache hit scenario, the MITM proxy server receives the cached response data from the API and **immediately returns it to the browser** as if it were the upstream server's reply. This means the browser gets the content without the request ever leaving our environment. Because the content came from our cache, we avoid any upstream latency and potentially reduce load on the external website. From the browser's perspective, it just received a valid response quickly; it does not know (nor need to know) that the content was served from cache. The proxy effectively **short-circuits** the request, acting as the server using cached data.

8. **Forward to Upstream on Cache Miss:** If no cached content was available (cache miss), the MITM proxy proceeds to forward the request to the **Upstream Web Server**. Before forwarding, the proxy can incorporate any header modifications that the API recommended (for example, adding an `X-Proxy-Cache: miss` header or other custom diagnostics, if we choose to). The request then goes out to the intended external host over the internet. From this point until the upstream responds, the MITM proxy simply waits, behaving like a normal forward proxy.

Response Handling and Caching Workflow

Once the upstream web server processes the request (in a cache miss scenario) and sends back a response, the MITM proxy intercepts that as well and triggers the response workflow:

1. **Upstream Response Received:** The MITM proxy server receives the HTTP response from the upstream server. This includes a status code (200, 404, etc.), response headers (Content-Type, Content-Length, Set-Cookie, etc.), and the response body (e.g. the HTML of a web page, JSON of

an API response, etc.). The proxy halts again before delivering this to the browser, and calls the `process_response` method on the MITM Proxy API Service, passing along the response data.

2. **API Service Processes the Response:** In `process_response`, our custom logic handles the received data. This stage is where we can implement content analysis, transformation, and caching. Key actions taken here:

3. **Caching the Content:** The response body (e.g., HTML of the page) and relevant metadata are sent to the **HTML Graph Service** to be stored. The service saves the raw HTML content into our cache store, so that future requests to the same URL can be served from this cache. Alongside storing the HTML, the system records a *flow entry* noting that this URL was fetched from upstream and cached at this time. It may also log the size of the content and how long the fetch took.
4. **Out-of-Band Processing:** Currently, any heavy content transformation or analysis is done out-of-band (i.e., not in real-time within the proxy's response cycle). For example, if we plan to run an HTML parser or analysis algorithm on the content, we would **trigger** that as a separate workflow or background job via the HTML Graph Service or another pipeline. In the current phase, `process_response` does not modify the content on the fly; it focuses on caching and logging. This design keeps the immediate response path fast.
5. **(Planned) In-Line Transformation:** In future iterations, we aim to incorporate inline transformation – meaning the `process_response` could alter the HTML or data before it's returned to the client (for instance, sanitizing content, injecting scripts, or annotating the page). The architecture is built to accommodate this, but initially we are deferring such transformations to maintain performance and stability.

6. **No Immediate Output to Proxy (for caching case):** In the present workflow, `process_response` typically doesn't need to send any modified content back to the proxy (unless we decide to modify something like headers or body on the fly). The proxy still holds the original upstream response. If we haven't altered it, the proxy will just use the original response for the client. The API service might return a confirmation or some data (like "cached:true") to the proxy, but not a new body.

7. **Deliver Response to Browser:** After `process_response` completes its tasks (which happen very quickly, since heavy lifting is deferred), the MITM proxy resumes the flow. It now forwards the upstream response (unaltered, in the current design) to the client browser. The browser receives the content it requested, unaware that it was intercepted and processed on the way. The end-user sees the page or resource load normally in their browser.

8. **Subsequent Requests – Cache Utilization:** With the content now cached in our system, if the user (or any user, if this is a shared cache) requests the same URL again, the request workflow (steps 1–3) will result in a cache hit. The `process_request` will detect the cached content and return it, enabling the proxy to serve the response immediately without contacting the upstream server. This demonstrates the full cycle of the caching mechanism:

9. The first request populates the cache (miss → fetch → store).
10. The second and further requests can be served from the cache (hit → no upstream call), greatly improving performance and reducing external calls.

Caching System and HTML Graph Service Details

Caching is a cornerstone of this platform. The **HTML Graph Service** is our custom caching and content management layer, which ensures that once a piece of content is fetched, it can be reused and analyzed.

- **Storage of HTML Content:** When a new page is fetched via the proxy (on a cache miss), the HTML Graph Service saves the page content in a persistent store. This could be a database, an object storage service, or a flat file store – implementation can be adjusted as needed. Each content blob is indexed by a key, typically a combination of the URL and possibly other factors (like a hash of the content or user/session if needed). In our demo, every unique page URL corresponds to an entry in the cache.

- **Flat HTML Domain:** We currently store the raw HTML (the full text of the webpage) as-is. This is referred to as the "flat HTML" storage – essentially the literal HTML source code of the page. Storing the exact response allows us to later serve it byte-for-byte to the browser on a cache hit, ensuring the content is identical to what the upstream server originally sent.

- **Content Retrieval:** The HTML Graph Service provides a lookup function for `process_request` to quickly retrieve content by URL. If found, the service returns the stored HTML along with any metadata (e.g., a timestamp, ETag, or content type). If not found, it signals a cache miss.

- **Logging and Flows:** Each significant action is logged as a *flow record* in the system. For example, a flow record is created when:

- A request is processed (noting URL, time, cache hit/miss).
- A response is processed and content is cached (noting content length, processing time, etc.).

- Any transformation or analysis step (in current or future workflows) is executed on the content. These flow records together form a history or graph of what happened to each request and piece of content. We can trace, for instance, that "Page A was requested at 10:00, was a cache miss, fetched from upstream in 200ms, stored in cache, then served from cache for 5 subsequent requests." The system captures timing information and outcomes at each stage, which is crucial for performance tuning and debugging.

- **Administration UI for Cache:** We have built an interface to visualize the cached data and flows:

- It lists cached pages (e.g., you might see entries like `/index.html`, `/about.html`, etc., corresponding to URLs fetched).
- Selecting a page shows the stored HTML content. We even have the ability to edit this content in the UI (for testing purposes). For example, one can modify the cached HTML and then refresh the page in the browser to immediately see the modified content, confirming that the cache is indeed being served instead of the upstream.

- The UI also surfaces the flow logs for each request. We can inspect how long each part of the pipeline took (e.g., time spent in `process_request`, time waiting for upstream, time in `process_response`, etc.), and any transformations that were applied. This provides insight into performance and correctness of the system.

- **Extensibility for Processing Flows:** The architecture is designed such that new *processing flows* can be added with minimal impact on the core proxy. For instance, we could introduce a flow that parses the HTML and extracts certain data, or a security scan that runs on the HTML. These would be implemented in the HTML Graph Service or a related service, and could be triggered upon storing new content (or updated content). Since currently these would run out-of-band, they would not slow down the user's request. Over time, as these processes mature, some could be moved inline if real-time transformation is desired.

Simulation and Testing Tools

To ensure the system works correctly and to facilitate development, we created a **Simulation and Testing framework** alongside the main system. This was necessary because testing a MITM proxy that requires a browser and external web pages can be cumbersome. The tools we built allow us to mimic the entire process in a controlled environment:

- **Request/Response Simulator:** This is a UI tool (and underlying API) that can simulate the behavior of the browser and upstream server towards the MITM Proxy API Service without needing the actual proxy in the middle. Essentially, the simulator sends crafted requests to `process_request` and `process_response` as if a real proxy were calling them. We can specify:
- The request details (URL, headers, cookies, etc.) to test how `process_request` handles them. The simulator will display what the API service returns — for example, whether it was a cache hit with a body, or a miss with instructions.
- The upstream response details (status code, headers, and body) to feed into `process_response`. This allows us to test how the system would handle various types of responses (HTML pages, JSON APIs, different sizes, etc.) and ensure the caching and processing logic works as expected.
- By chaining these, we can simulate an entire flow: first call `process_request` for a URL (get a miss), then simulate an upstream response for that URL, and call `process_response`. The next simulation of `process_request` for the same URL should then yield a cache hit (with a cached body).

- This simulator has been crucial for debugging. For example, it helped verify that when a page is cached, a subsequent request returns the cached content immediately without going upstream. It also allows testing of header modifications: we can see if our API is instructing the proxy to add or remove certain headers on the fly.

- **Automated Load Testing:** Because the API service is independent of the actual proxy, we can write automated tests or scripts to stress-test the system. Using the simulator or direct calls, we can simulate high volumes of requests to see how the `process_request` and `process_response` logic perform under load. This is useful for **performance testing at each layer**:

- We can test the **MITM Proxy API Service** in isolation (simulate many requests without involving actual network calls to upstream). This tells us how our Python service and the caching backend scale (e.g., how many requests per second it can handle, how it behaves with increasing cache size).
- We can test the **full pipeline** end-to-end by using real browser instances or scripts hitting the proxy with actual web traffic to see overall throughput and latency.

- By comparing these, we can identify if any layer is the bottleneck (for instance, if the proxy server itself adds significant overhead, or if the caching database becomes slow, etc.). In our observations so far, the core proxy (written in a low-level, optimized way) is very efficient and not the limiting factor. It appears to handle intercepting traffic with minimal latency, and we expect it to scale linearly with additional instances (we can run multiple proxy containers/instances behind a load balancer if needed).

- **Use Case Simulation (Costing and Traffic Patterns):** The ability to simulate requests means we can model different usage scenarios. For example, we can simulate a heavy user load or frequent refreshes of certain pages to see how the cache hit/miss ratio behaves, which in turn can inform us about bandwidth usage and potential cost of upstream data. The meeting discussion mentioned using the simulation for "costing" – i.e., to estimate how much external bandwidth or compute is saved by caching, or how the system would handle a very high volume of traffic without deploying a full fleet of browsers and proxies. This gives us insight into potential cloud costs and helps in capacity planning.

In summary, our testing tools ensure that we don't need to manually open a browser and click refresh repeatedly to validate the system's behavior. We have full programmatic control to simulate scenarios, which speeds up development and testing significantly. The **API service is oblivious to whether the traffic is real or simulated**, which speaks to the transparency and modularity of our design.

Performance and Scalability Considerations

Designing a high-performance intercepting proxy system required careful attention to each component's efficiency and scalability:

- **MITM Proxy Server Performance:** The proxy component we use is proven and efficient. During development, we observed that it handles request interception and forwarding with very low overhead. It's built on robust networking libraries, making it capable of handling many simultaneous connections. We term it "battle-hardened" because it has been used in similar scenarios before and optimized for performance. Additionally, since the proxy server is stateless (it relies on the external API and cache for logic and data), we can scale horizontally by running multiple instances. Each instance can be deployed on a separate EC2 machine or container, and we could distribute traffic among them if needed for load balancing.

- **API Service and Caching Performance:** The MITM Proxy API Service does introduce some overhead (network calls between the proxy and API, cache lookups, etc.), but this is designed to be as lightweight as possible:

- Cache lookups on `process_request` are typically O(1) operations in our storage (keyed access to a database or in-memory store).
- Writing to the cache in `process_response` is done asynchronously or quickly enough that it hasn't been a bottleneck. If the storage layer is slow, we have the option to batch or queue writes.
- The API's processing logic is simple for now (just decisions and storing data). As we add more complex processing, we will measure their impact. We have the flexibility to offload heavy computation to background jobs or specialized services to keep the API responsive.

- The use of Python for the API service is convenient for development, but we are mindful of efficiency. If needed, critical sections can be optimized or moved to a faster language, or the service can be scaled out behind a load balancer as well.

- **Asynchronous Processing:** A key performance strategy in our architecture is doing expensive tasks outside the critical path. By not modifying or scanning the HTML in real-time (for now), we ensure the user's request is fulfilled as fast as possible – essentially the added latency is just the cache check and a database write, which are minimal. Any heavy analysis (like parsing the HTML to build a DOM graph, running security checks, etc.) can be done after the response is sent to the browser. This way, the user experience remains snappy. In future, if inline processing is needed, we will only incorporate it if we can do so efficiently (or perhaps make it optional based on content type or user preferences).

- **Throughput and Bottleneck Analysis:** We plan and have started to conduct performance tests at each layer:

- Testing the proxy alone with dummy endpoints (to ensure it can handle X requests/sec).
- Testing the API and cache alone (simulate rapid `process_request` / `process_response` calls).

- Testing end-to-end with actual web pages.
  Early indications show the proxy and API can handle the load for our current needs. The detailed flow logs help pinpoint where any slowdowns occur. For instance, if a particular site's pages are very large, we might see the caching write taking more time; this might suggest using streaming or chunked processing in the future.

- **Scalability:** Each component can be scaled:

- The **MITM proxy** is stateless and can be replicated. We could have multiple proxies behind a routing layer; all of them could point to the same API service and caching backend.
- The **API service** could also be replicated if needed, with proper load balancing and a shared backing store (database) for the cache. It's a RESTful service that can be scaled horizontally.
- The **HTML Graph (storage)** might need scaling or partitioning if the volume of stored content becomes very large. Since it behaves like a database, we would ensure it is a scalable one (NoSQL or distributed file store as needed).

- Our design cleanly separates concerns, which means each piece can be tuned or scaled independently. For example, if caching becomes the bottleneck, we can upgrade the database or use an in-memory cache. If network I/O is the bottleneck, more proxy instances can be added.

- **Reliability:** By intercepting traffic, the proxy effectively becomes a critical component in the browsing workflow. We are taking measures to ensure reliability:

- Timeouts and fallbacks: If the API service were ever unresponsive, the proxy should have a sane fallback (e.g., bypass the cache and go directly upstream to avoid stalling the user).
- Error handling: The system should handle cases where the cache storage is down or a content parsing flow fails, without breaking the user's browsing. Typically, the user would just get the original content (maybe slightly delayed) even if our processing fails.
- Monitoring: The detailed logs and UI help monitor the health and performance. Any anomalies in processing times or error rates can be spotted quickly.

Conclusion

In this document, we presented a detailed overview of our Man-in-the-Middle Proxy platform that intercepts and processes web requests in real-time. The architecture is modular, consisting of a robust intercepting proxy and a flexible API-driven processing layer with a dedicated caching service. We described how requests flow through the system: from the browser to the proxy, through the processing service, to the upstream server, and back – with caching strategically integrated to improve efficiency. We also highlighted the tools we built for simulation, testing, and monitoring, which have been critical in refining the system.

This platform effectively allows us to **insert intelligent processing into web traffic** without requiring changes on the client or server side. By doing so, we can cache content for faster load times, analyze pages on the fly, and prepare for more advanced transformations or analyses (via the HTML Graph concept) in the future. The white-paper style briefing above should serve as a reference for our team and stakeholders to understand the technical workings and benefits of the system.

Moving forward, we plan to leverage this architecture for various use cases, such as content augmentation, security scanning, or analytics, by adding new processing flows. We will continue to conduct performance testing and optimizations at each layer to ensure scalability as usage grows. With its solid foundation and flexible design, the MITM proxy platform is well-positioned to support our current objectives and expand with future requirements.
